Case Study: Watchdog Timer (WDT) Functionality on Arduino Nano 33 BLE Sense

1 Objective

To design, implement, and demonstrate the working of the Watchdog Timer (WDT) peripheral on the Arduino Nano 33 BLE Sense board (Nordic nRF52840 SoC), by intentionally simulating a firmware hang/freeze condition and observing that the WDT automatically resets the microcontroller when the system fails to periodically ”feed” (reset) the timer within the configured timeout window.

2 Introduction

A Watchdog Timer (WDT) is a hardware timer that is used to detect and recover from software or hardware malfunctions in embedded systems. During normal operation, the application code must periodically reset (“feed” or “kick”) the WDT counter. If the counter is not reset within a predefined timeout interval – typically because the main program has hung, entered an infinite loop, or become unresponsive due to a bug – the WDT triggers an automatic system reset, allowing the device to recover without manual intervention.

The Arduino Nano 33 BLE Sense is built around the Nordic Semiconductor nRF52840 SoC, which includes a dedicated hardware WDT peripheral. This case study uses the NRF WDT register-level driver (accessible through the Mbed / nRF5 core bundled with the Arduino mbed nano board package) to configure and test this functionality.

3 HardwareandSoftwareRequirements

- Arduino Nano 33 BLE Sense (Rev1 or Rev2)
- Micro-USB cable
- Arduino IDE (2.x recommended) with the Arduino Mbed OS Nano Boards core installed
- Serial Monitor (115200 baud)
- On-board LED (pin 13 / LED BUILTIN) for visual status indication

4 WorkingPrinciple

1. On boot, the sketch prints the reset reason (to distinguish a normal power-on reset from a watchdog-triggered reset).

2. The WDT is configured with a timeout of 3 seconds using the NRF WDT register interface.

3. In the loop(), the program feeds the watchdog every 1 second under normal operation, blinking the on-board LED to indicate healthy operation.

4. After 10 feed cycles (˜10 seconds), the code deliberately enters an infinite while(1) loop to simulate a firmware hang/freeze – the watchdog is no longer fed.

5. Since the WDT is not reloaded within its 3-second timeout window, the nRF52840 automatically resets itself.

6. Upon reboot, the sketch detects the RESETREAS register flag corresponding to a watchdog reset and prints a confirmation message, along with a distinct fast-blink LED pattern – visually proving that the WDT triggered the recovery.

5 ArduinoSketch(WDTNano33BLE.ino)

```
/*

  Watchdog Timer (WDT) Demonstration
  Board   : Arduino Nano 33 BLE Sense (nRF52840)
  Purpose : Configure hardware WDT, feed it periodically, then
            simulate a firmware hang to trigger an automatic reset.
*/

#include <nrf.h>

#define WDT_TIMEOUT_MS 300    // 3-second WDT timeout
#define FEED_INTERVAL_M 0     // Feed WDT every 1 second
#define S               100   // Simulate hang after 10 feeds (~10 s)
        HANG_AFTER_FEE  0 10
        DS
uint32_t feedCount = 0;

void configureWDT(uint32_t timeoutMs) {
  NRF_WDT->CONFIG = (WDT_CONFIG_HALT_Pause << WDT_CONFIG_HALT_Pos)
                  | (WDT_CONFIG_SLEEP_Run << WDT_CONFIG_SLEEP_Pos);
  NRF_WDT->CRV = (timeoutMs * 32768ULL) / 1000;   // 32.768 kHz clock
  NRF_WDT->RREN |= WDT_RREN_RR0_Msk;               // Enable reload
   register 0
  NRF_WDT->TASKS_START = 1;                        // Start the WDT
}

void feedWDT() {
  NRF_WDT->RR[0] = WDT_RR_RR_Reload;
}

bool wasResetByWDT() {
  return (NRF_POWER->RESETREAS & POWER_RESETREAS_DOG_Msk) != 0;
}
```

```
31
32 void clearResetReason() {
33   NRF_POWER->RESETREAS = NRF_POWER->RESETREAS; // Write 1 to clear
       flags
34 }
35
36 void setup() {
37   Serial.begin(115200);
38   while (!Serial && millis() < 3000);
39
40   pinMode(LED_BUILTIN, OUTPUT);
41
42   Serial.println("========================================");
43   Serial.println("  Nano 33 BLE Sense - WDT Case Study Demo  ");
44   Serial.println("========================================");
45
46   if (wasResetByWDT()) {
47     Serial.println(">>> SYSTEM RECOVERED FROM A WATCHDOG RESET! <<<");
48     Serial.println(">>> WDT functionality CONFIRMED working. <<<");
49     // Fast blink pattern = proof of WDT recovery
50     for (int i = 0; i < 10; i++) {
51       digitalWrite(LED_BUILTIN, HIGH); delay(100);
52       digitalWrite(LED_BUILTIN, LOW); delay(100);
53     }
54   } else {
55     Serial.println("Normal power-on / manual reset detected.");
56   }
57
58   clearResetReason();
59   configureWDT(WDT_TIMEOUT_MS);
60   Serial.println("WDT configured with 3000 ms timeout.");
61   Serial.println("Feeding WDT every 1000 ms...");
62 }
63
64 void loop() {
65   feedWDT();
66   feedCount++;
67
68   digitalWrite(LED_BUILTIN, HIGH);
69   delay(150);
70   digitalWrite(LED_BUILTIN, LOW);
71
72   Serial.print("WDT fed. Count = ");
73   Serial.println(feedCount);
74
75   if (feedCount >= HANG_AFTER_FEEDS){
76     Serial.println("Simulating firmware hang (infinite loop)...");
77     Serial.println("WDT will NOT be fed from this point onward.");
78     while (1) {
79       // Deliberate infinite loop - no feedWDT() call here.
80       // The nRF52840 WDT will expire after ~3 s and force a reset.
81     }
82   }
83
84   delay(FEED_INTERVAL_MS - 150);
85 }
```

Listing 1: Watchdog Timer demonstration sketch for Nano 33 BLE Sense

6 Serial Monitor Output

```
1  ========================================
2   Nano 33 BLE Sense - WDT Case Study Demo
3  ========================================
4  Normal power-on / manual reset detected.
5  WDT configured with 3000 ms timeout.
6  Feeding WDT every 1000 ms...
7  WDT fed. Count = 1
8  WDT fed. Count = 2
9  ...
10 WDT fed. Count = 10
11 Simulating firmware hang (infinite loop)...
12 WDT will NOT be fed from this point onward.
13
14 [ ~3 seconds later, board auto-resets ]
15
16 ========================================
17  Nano 33 BLE Sense - WDT Case Study Demo
18 ========================================
19 >>> SYSTEM RECOVERED FROM A WATCHDOG RESET! <<
20 WDT WDT functionality CONFIRMED working. <
21 Feeding configured with 3000 ms timeout. <<
22         WDT every 1000 ms... <
23 WD fed. Count = 1
24 T
...
```

Listing 2: Serial output across the hang-reset cycle

7 Observations

- During normal operation, the on-board LED blinks once per second, confirming that the main loop is alive and feeding the watchdog.
- After 10 feed cycles, the sketch enters a deliberate infinite loop, halting all further watchdog feeds.
- Within the configured 3-second timeout, the nRF52840's WDT peripheral forces a full system reset, independent of software state.
- On reboot, the RESETREAS register correctly reports a watchdog-induced reset, which the sketch detects and reports both over Serial and via a fast LED blink
- pattern.
  The board is capable of running this hang-and-recover cycle indefinitely without manual intervention, proving autonomous fault recovery.

8 Statement

"This case study demonstrates that the Arduino Nano 33 BLE Sense, powered by the Nordic nRF52840 SoC, successfully implements hardware Watchdog Timer (WDT) func- tionality. By configuring the WDT with a 3-second timeout and deliberately simulating a firmware hang after 10 seconds of normal operation, it was experimentally verified that themicrocontrollerautonomouslydetectstheunresponsivestateandperformsahardware

reset without any external intervention. Post-reset analysis of the RESETREAS register confirmedtheresetwaswatchdog-triggered,validatingthattheWDTperipheralfunctions correctly as a fault-tolerance and self-recovery mechanism, which is essential for unattended, mission-critical, and IoT embedded deployments.”

9 Applications

- Autonomous recovery in remote/unattended IoT nodes
- Fault tolerance in medical and industrial embedded devices
- Preventing permanent system lock-up due to software bugs or transient hardware glitches
- Safety-critical systems requiring guaranteed uptime

10 Project Links

- GitHub Repository: https://github.com/ak4204/nano33ble-wdt-case-study/

11 Conclusion

The experiment successfully validates the Watchdog Timer functionality of the Arduino Nano 33 BLE Sense. The hardware-level WDT implementation ensures that the sys- tem can recover autonomously from software hangs, making it a reliable mechanism for building robust and fault-tolerant embedded applications.